

Computergrafik

PD Prof. Dr. rer nat.
Marco Block-Berlitz

Sommersemester 2026



OpenGL-Grundlagen: Dreiecke, Interpolation und Rendering-Pipeline

„Nichts ist so beständig wie der Wandel.“ (Heraklit von Ephesus)

PD Prof. Dr. Marco Block-Berlitz

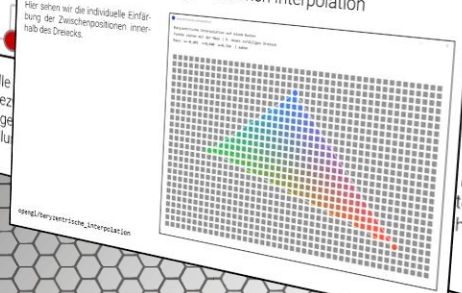
OpenGL als Zustandsautomat

OpenGL wurde als **Zustandsautomat** realisiert. Es gibt eine Vielzahl von Parametern, die beschreiben, wie OpenGL mit vorhandenen Daten arbeiten soll. Das bedeutet, dass alle Parameter so lange Anwendung finden, bis sie verändert werden.



Interaktives Beispiel zur baryzentrischen Interpolation

Hier sehen wir die individuelle Einfärbung der Zwischenpositionen innerhalb des Dreiecks.



Stellen wir beispielsweise die aktuelle Methode wieder die Standardeinstellung

Durch
ten zu
hende

10

Ziele und Inhalt

- Grafik-APIs, OpenGL, Vulkan, DirectX, Metal
- OpenGL in Processing und LWJGL
- OpenGL als Zustandsautomat
- Objektrepräsentationen
- Dreiecke unter der Lupe
- Baryzentrische Interpolation
- Geometrischer Baukasten (Primitive)
- Vereinfachte Renderpipeline in OpenGL
- Zusammenspiel von OpenGL und LWJGL
- Ein Dreieck in OpenGL konstruieren
- Bewegtes Dreieck realisieren
- LWJGLBasisFenster anpassen
- AWT/Swing und LWJGL zusammenbringen
- LWJGL-Kontext in Canvas einbetten
- Hintergrund mit ToggleButton umschalten
- Transformationsketten in OpenGL
- Primitives for OpenGL (POGL)
- 3D-Würfel konstruieren
- Würfelprojektion in Fläche
- Wehende Fläche
- Fragestellungen zum Vorlesungsteil
- Praktische Aufgaben





Moderne Computergrafik braucht eine direkte
Schnittstelle zur Hardware

Grafik-APIs in der modernen Computergrafik

Moderne Grafikhardware ist extrem komplex. Eine GPU besteht aus tausenden Recheneinheiten, die parallel arbeiten. Damit Anwendungen diese Hardware nutzen können, gibt es [Grafik-Programmierschnittstellen](#) (APIs).

Diese APIs abstrahieren die GPU und stellen [Funktionen](#) bereit für:

- Rendering von Geometrie
- Shader-Programmierung
- Speicherverwaltung
- Texturen und Beleuchtung

Wichtige [Grafik-APIs](#) heute:

- OpenGL (plattformübergreifend)
- DirectX (Windows, Xbox)
- Vulkan (plattformübergreifend)
- Metal (Apple)

Grafik-APIs bilden die [Schnittstelle zwischen Software und GPU](#).

OpenGL versus Vulkan API

Klassische Grafik-APIs wie [OpenGL](#) übernehmen sehr viel Arbeit automatisch. Zum Beispiel: Speicher-verwaltung, Synchronisation oder die Pipeline-Konfiguration. Das ist bequem, hat aber einen Nachteil:

Die Treiber müssen viel Arbeit leisten.

Der [moderne Nachfolger Vulkan](#) (seit 2016) gibt diese Kontrolle stärker an den Entwickler zurück. Das führt zu höherer Performance, besserer Nutzung moderner GPUs aber auch deutlich [komplexerer Programmierung](#).

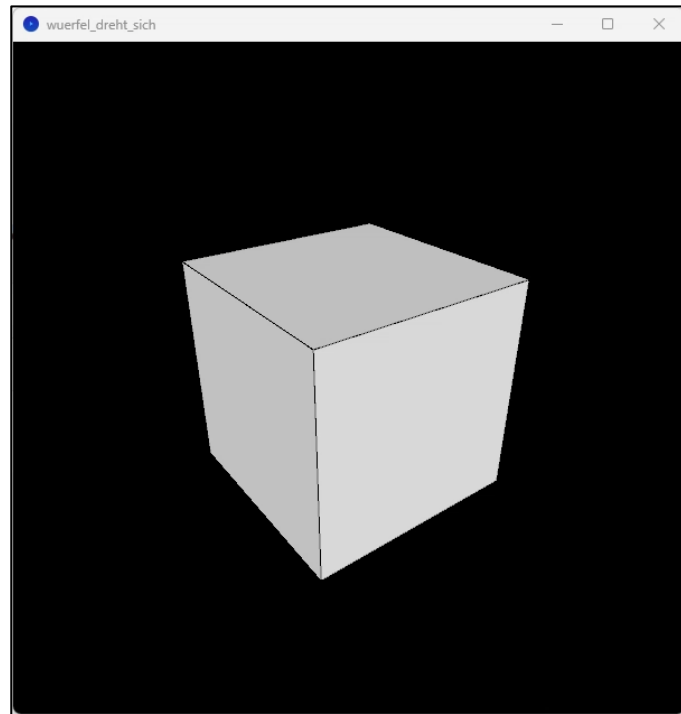
Vulkan ist nicht Bestandteil dieser Vorlesung. Wir verwenden OpenGL, weil es deutlich einfacher zu erlernen ist.

OpenGL in Processing und LWJGL

Wir werden OpenGL in verschiedenen Programmiersprachen und Frameworks erleben.

```
void setup() {  
  size(600, 600, P3D);  
}  
  
void draw() {  
  background(0);  
  lights(); // aktiviert Standard-Beleuchtung  
  
  translate(width/2, height/2, 0); // Szene zentrieren  
  
  rotateX(-PI/6); // leichte Neigung (Blick von oben)  
  rotateY(frameCount * 0.02); // Rotation des Würfels  
  
  box(200);  
}
```

Processing

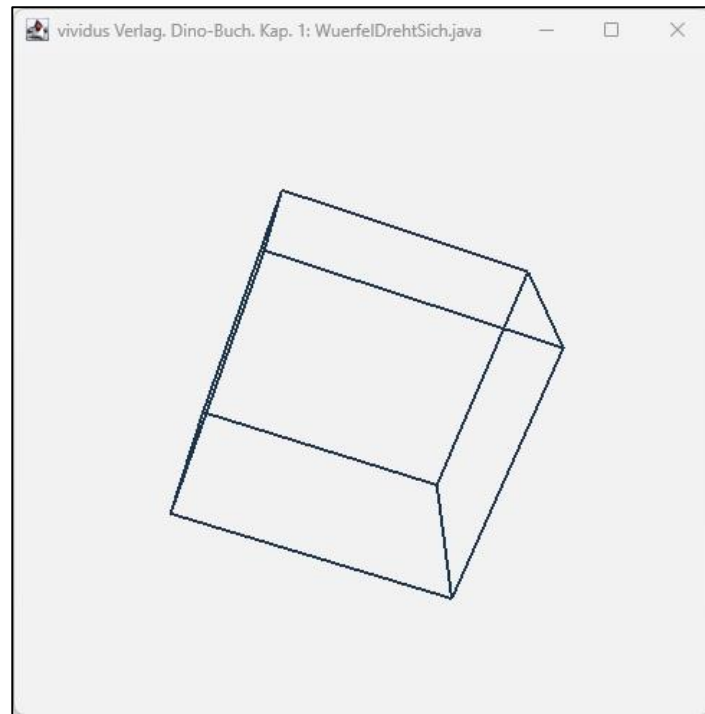


OpenGL in Processing und LWJGL

Wir werden OpenGL in verschiedenen Programmiersprachen und Frameworks erleben.

```
glPolygonMode(GL_FRONT_AND_BACK, GL_LINE);  
long start = System.nanoTime();  
while (!Display.isCloseRequested()) {  
    double t = (System.nanoTime() - start)/1e9;  
    POGI.clearBackgroundWithColor(0.95f, 0.95f, 0.95f, 1.0f);  
  
    glLoadIdentity();  
    glFrustum(-1, 1, -1, 1, 4, 10);  
    glTranslated(0, 0, -5);  
    glRotatef((float)t*100.0f, 1.0f, 0.0f, 0.0f);  
    glRotatef((float)t*100.0f, 0.0f, 1.0f, 0.0f);  
    glScaled(0.5, 0.5, 0.5);  
    glLineWidth(2.0f);  
    glColor3d(0.1, 0.2, 0.3);  
    POGI.renderWuerfel();  
  
    Display.update();  
}
```

Java

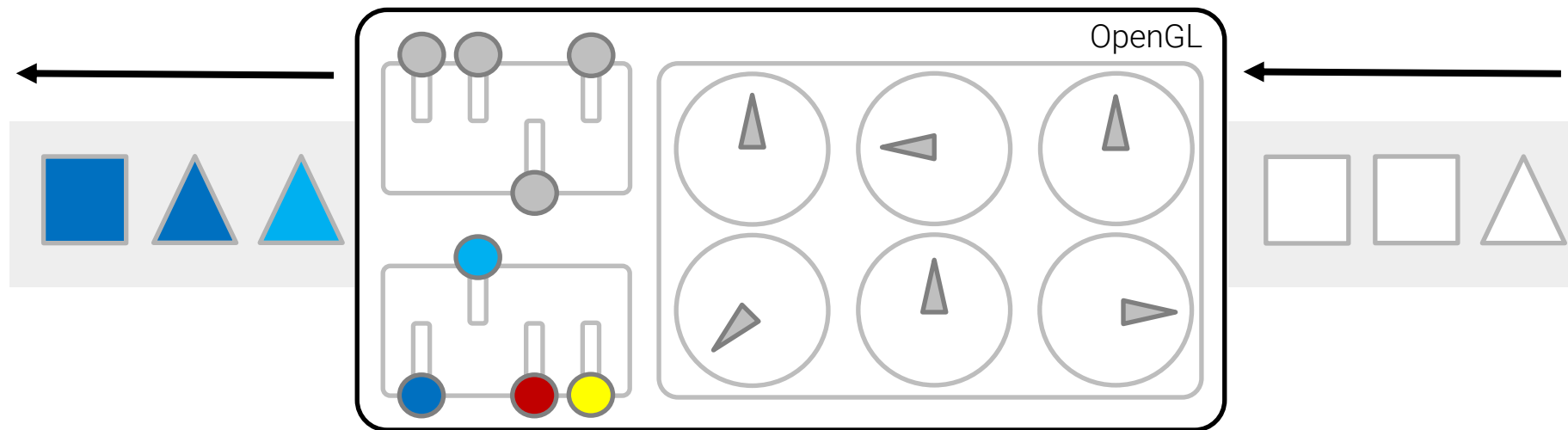




Zunächst ein paar einleitende Worte zu [OpenGL](#)

OpenGL als Zustandsautomat

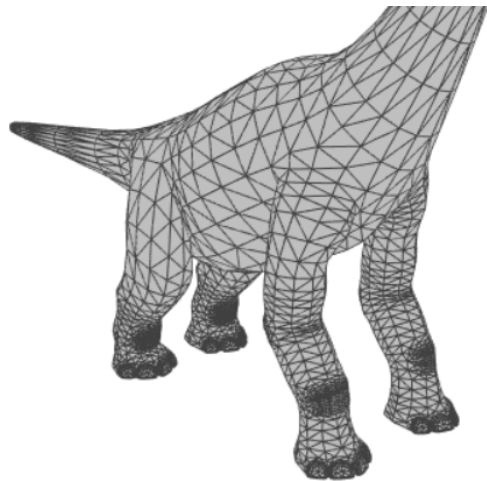
OpenGL wurde als [Zustandsautomat](#) realisiert. Es gibt eine Vielzahl von Parametern, die beschreiben, wie OpenGL mit vorhandenen Daten arbeiten soll. Das bedeutet, dass alle Parameter so lange Anwendung finden, bis sie verändert werden.



Stellen wir beispielsweise die aktuelle Farbe auf Hellblau, werden im Anschluss alle Vertices in Hellblau dargestellt. Durch dieses Konzept werden wir dazu gezwungen, uns mit den [Standardeinstellungen](#) des Zustandsautomaten zu beschäftigen. Um Programmpassagen austauschbar zu machen, sollten im Anschluss an die entsprechende Methode wieder die Standardeinstellungen verwendet werden.

Objektrepräsentationen

Zunächst wollen wir mit der Geometrie starten. Um Objekte in geeigneter Weise in einer 3D-Umgebung zu repräsentieren, stehen uns verschiedene Möglichkeiten zur Verfügung. Links: Variante mit einer **triangulierten Oberfläche**, die später mit Texturen gefüllt wird. Rechts: Bei der Wahl der **parametrisierten Repräsentation** kommen sehr unterschiedliche geometrische Modelle zum Einsatz.



Triangulierte Oberfläche



3D-Objekt



Parametrisierte Oberfläche



Auch wenn es zunächst verwunderlich klingen mag,
wollen wir Oberflächen von Objekten durch verkettete
Dreiecke oder **Vierecke** beschreiben.

Dreiecke unter der Lupe

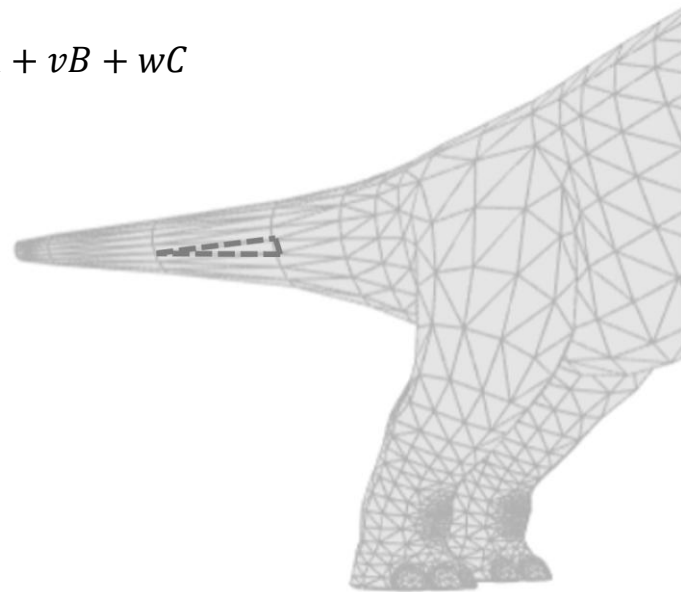
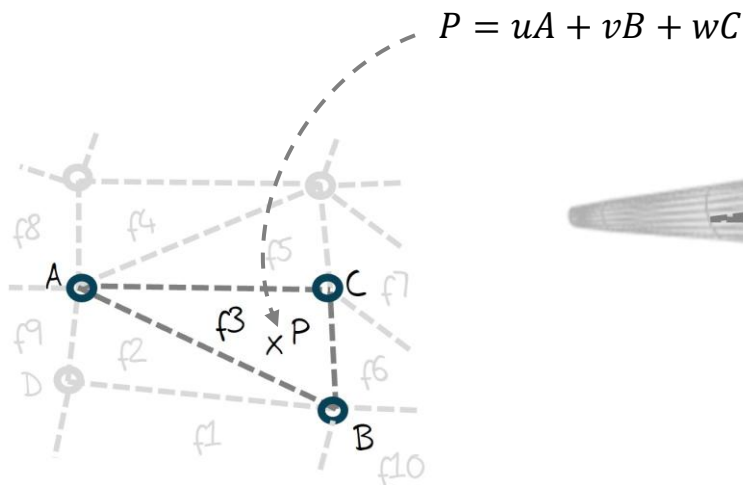
Objekte werden in der Regel durch **Dreiecke** repräsentiert. Eine triangulierte Oberfläche, die aus vernetzten Dreiecken besteht, kann an den Knoten neben den relativen x -, y - und z -Koordinaten auch Informationen zu Farben oder anliegenden Texturen speichern.

Vertex-Liste

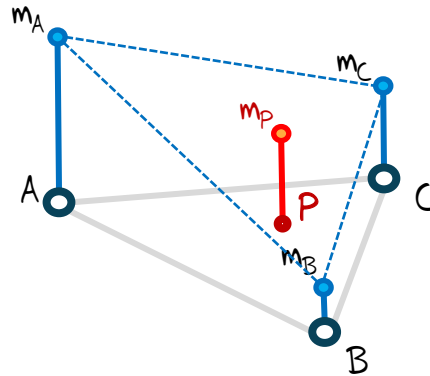
```
...  
A 0.0033 0.0057 0.1845 f2 f3 f5 f4 f8 f9  
B 0.0051 0.0058 0.1920 f2 f1 f10 f6 f3  
C 0.0053 0.0045 0.2010 f3 f6 f7 f5  
...
```

Face-Liste

```
...  
f2 A D B  
f3 A B C  
...
```



Wir haben jetzt folgende Situation zu lösen:



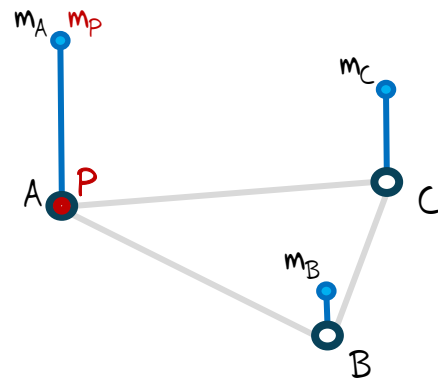
Für die **Interpolation** gibt es drei Fälle zu betrachten.

Baryzentrische Interpolation – Fall 1: Punkt

Wir landen mit dem gesuchten Punkt direkt auf einem der Eckpunkte. In diesem Fall landet P direkt auf A . Gegeben sind $P = (x_P, y_P)$, $A = (x_A, y_A)$ und m_A und gesucht ist m_P .

Lösung

Wenn P auf A fällt $\Rightarrow m_P = m_A$.



Baryzentrische Interpolation – Fall 2: Gerade

Wir landen mit dem gesuchten Punkt direkt auf einer der Geraden zwischen zwei Eckpunkten. In diesem Fall landet P direkt auf der Geraden zwischen A und B .

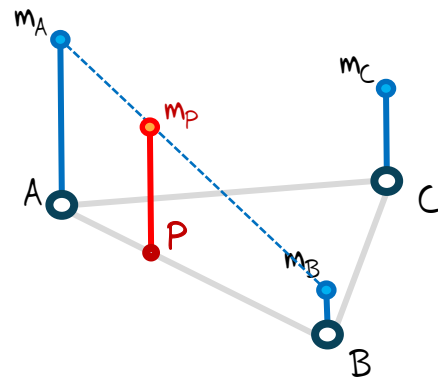
Gegeben sind $P = (x_P, y_P)$, $A = (x_A, y_A)$, $B = (x_B, y_B)$, m_A und m_B und gesucht ist m_P .

Idee: Wir betrachten das Verhältnis von P zu A und P zu B in Bezug auf die Länge der Gesamtstrecke $\overline{AB}$. Es gilt $P = uA + vB$ und daher bestimmen wir zunächst u und v .

Anschließend können wir m_P über $m_P = um_A + vm_B$ leicht bestimmen.

Lösung

$$u = \frac{\|P-B\|}{\|B-A\|} \text{ und } v = \frac{\|P-A\|}{\|B-A\|}$$



Baryzentrische Interpolation – Fall 3: Dreieck

Wir landen mit dem gesuchten Punkt innerhalb des Dreiecks ΔABC . Gegeben sind $P = (x_P, y_P)$, $A = (x_A, y_A)$, $B = (x_B, y_B)$, $C = (x_C, y_C)$, m_A , m_B und m_C und gesucht ist m_P .

Idee: Wir stellen ein Gleichungssystem auf und lösen es mit Hilfe der Cramerschen Regel unter Einbehaltung der Normierungseigenschaft $u + v + w = 1$. Es gilt $P = uA + vB + wC$ und daher bestimmen wir zunächst u , v und w . Anschließend können wir m_P über $m_P = um_A + vm_B + wm_C$ wieder leicht bestimmen.

Lösung

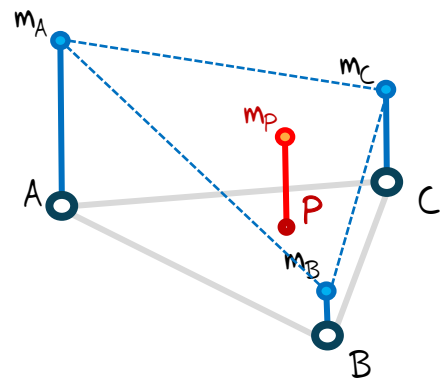
$$u = \frac{(x_B - x_P)(y_C - y_P) - (x_C - x_P)(y_B - y_P)}{(x_B - x_A)(y_C - y_B) - (y_B - y_A)(x_C - x_B)}$$

$$v = \frac{(x_C - x_P)(y_A - y_P) - (x_A - x_P)(y_C - y_P)}{(x_B - x_A)(y_C - y_B) - (y_B - y_A)(x_C - x_B)}$$

$$w = \frac{(x_A - x_P)(y_B - y_P) - (x_B - x_P)(y_A - y_P)}{(x_B - x_A)(y_C - y_B) - (y_B - y_A)(x_C - x_B)}$$

Da $u + v + w = 1$ gilt, genügt es zwei der drei Unbekannten zu bestimmen:

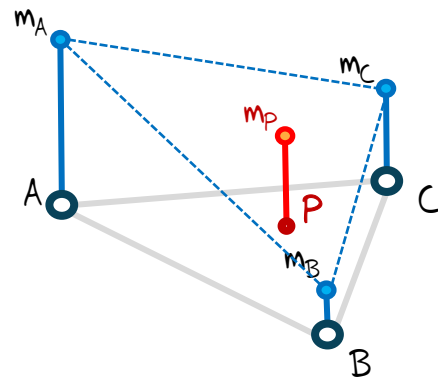
$$w = 1 - (u + v)$$



Baryzentrische Interpolation – Implementierung

Schauen wir uns eine Implementierungsvariante dazu an:

```
public static double barycentricInterpolation(Vektor2D P, Vektor2D A,  
    Vektor2D B, Vektor2D C, double w_a, double w_b, double w_c) {  
    double denom = 1./((B.x-A.x) * (C.y-B.y) - (B.y-A.y) * (C.x-B.x));  
    double u = ((B.x-P.x) * (C.y-P.y) - (C.x-P.x) * (B.y-P.y)) * denom;  
    double v = ((C.x-P.x) * (A.y-P.y) - (A.x-P.x) * (C.y-P.y)) * denom;  
    double w = 1. - u - v;  
  
    return u*w_a + v*w_b + w*w_c;  
}
```



So könnte ein Beispielaufruf aussehen:

```
Vektor2D P = new Vektor2D(7,5);  
Vektor2D A = new Vektor2D(1,3);  
Vektor2D B = new Vektor2D(10,6);  
Vektor2D C = new Vektor2D(12,2);  
double w_a=10, w_b=8, w_c=4;  
  
double w_p = barycentricInterpolation(P, A, B, C, w_a, w_b, w_c);  
System.out.println(w_p);
```

Java

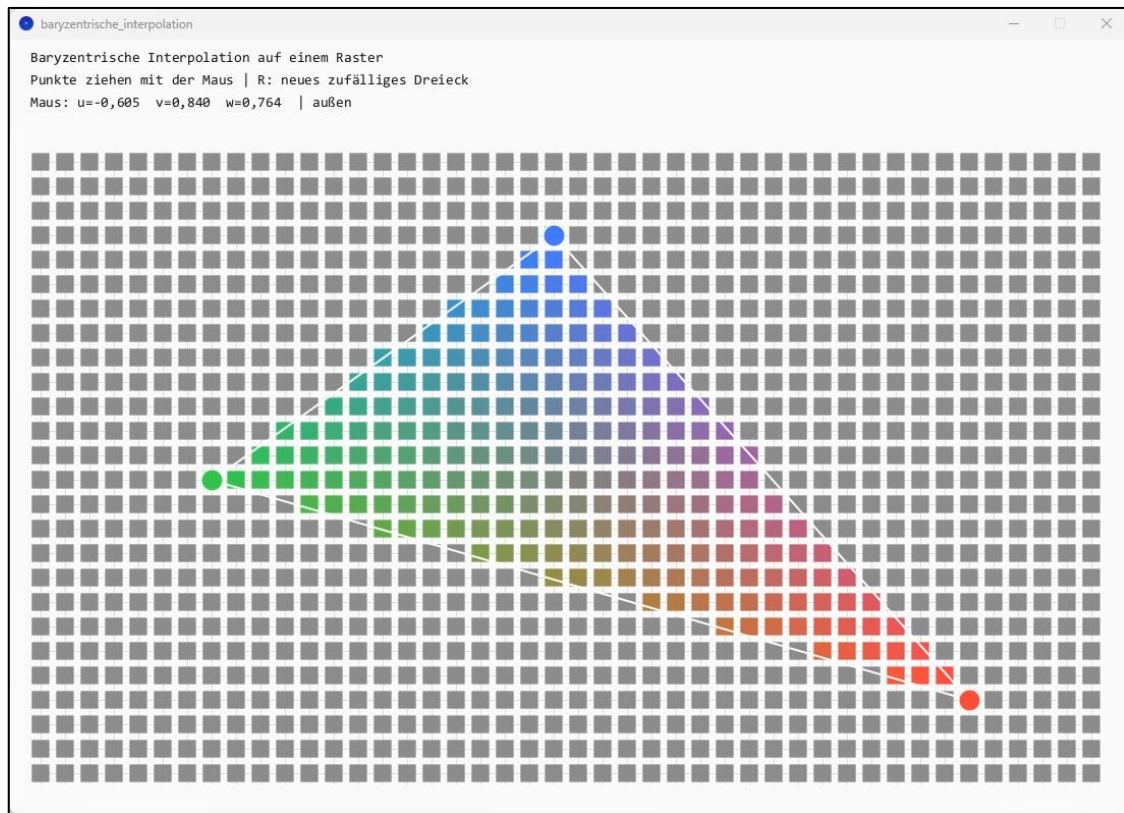
$$u = \frac{(x_B - x_P)(y_C - y_P) - (x_C - x_P)(y_B - y_P)}{(x_B - x_A)(y_C - y_B) - (y_B - y_A)(x_C - x_B)}$$

$$v = \frac{(x_C - x_P)(y_A - y_P) - (x_A - x_P)(y_C - y_P)}{(x_B - x_A)(y_C - y_B) - (y_B - y_A)(x_C - x_B)}$$

$$w = \frac{(x_A - x_P)(y_B - y_P) - (x_B - x_P)(y_A - y_P)}{(x_B - x_A)(y_C - y_B) - (y_B - y_A)(x_C - x_B)}$$

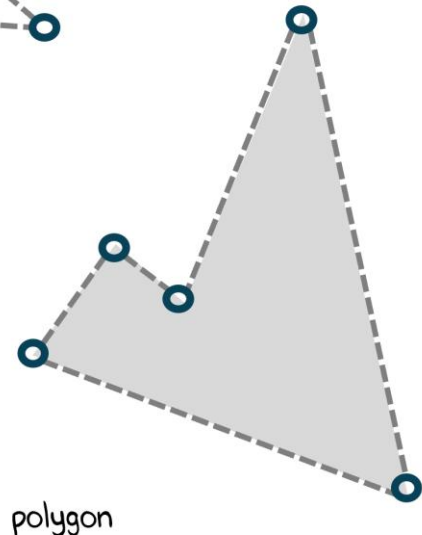
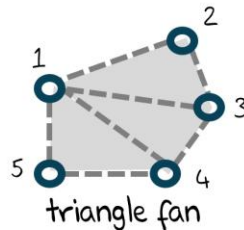
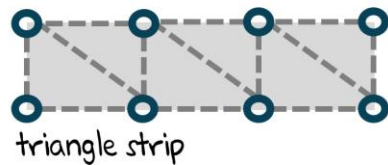
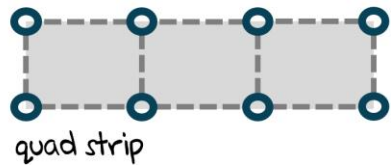
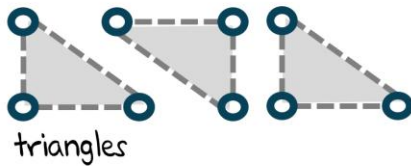
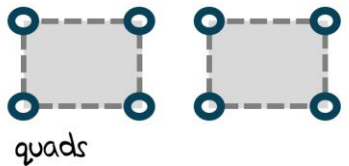
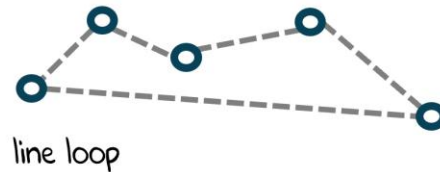
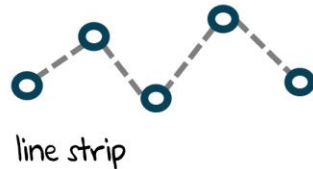
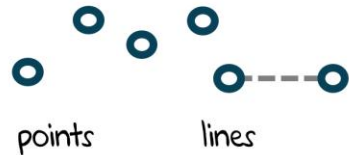
Interaktives Beispiel zur baryzentrischen Interpolation

Hier sehen wir die individuelle Einfärbung der Zwischenpositionen innerhalb des Dreiecks.



Geometrischer Baukasten

Ein Baukasten aus zehn kleinen geometrischen Bausteinen, die wir **Primitive** nennen, steht uns in OpenGL zur Verfügung. Ein 3D-Modell besteht aus zusammengesetzten Primitiven und wird anschließend texturiert, indem die Primitiven Teilabschnitte einer gegebenen Oberfläche repräsentieren. Für das Rendering einer 3D-Szene werden die 3D-Objekte entsprechend der gewünschten Ausrichtung und Größe positioniert.

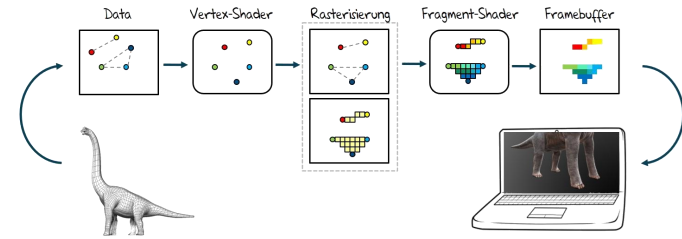




Bevor wir tiefer in OpenGL einsteigen, wollen wir uns die vereinfachte Renderpipeline anschauen.

Vereinfachte Renderpipeline – Phase 1

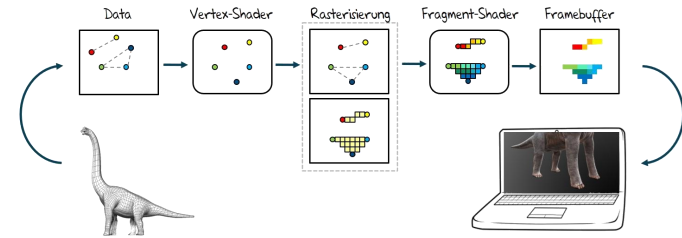
In der ersten Phase werden die Daten der Primitiven, bestehend aus Vertices, definiert (**Data**). In diesem Beispiel trägt jeder Vertex, symbolhaft für eine ganze Reihe von möglichen Informationen, eine unterschiedliche Farbe. Die Primitiven haben jeweils unterschiedliche Bezugssysteme.



Vereinfachte Renderpipeline – Phase 2

Um diese Koordinatensysteme in der darzustellenden Welt zu vereinen, müssen sie transformiert werden. Diese Aufgabe übernimmt der **Vertex-Shader**. Jedem Vertex wird dabei eine Position in der zu generierenden Welt zugewiesen, es können sogar mehrere auf dieselbe Position fallen. Dabei spielen die Informationen über die dazugehörigen Primitiven zunächst keine Rolle.

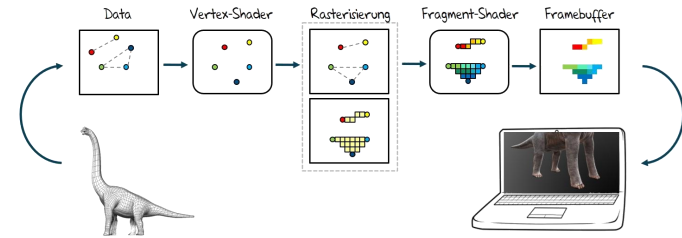
Alle Vertices werden quasi **parallel** transformiert. Sie kennen also die neuen Positionen der Nachbarknoten zu diesem Zeitpunkt nicht.



Vereinfachte Renderpipeline – Phase 3

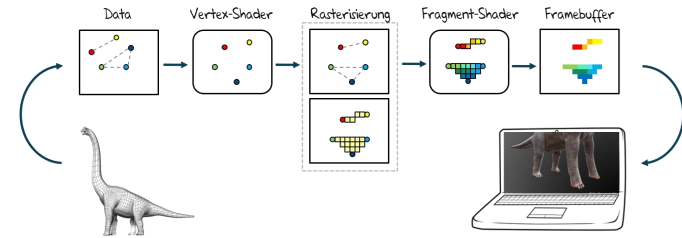
Im anschließenden **Rasterisierungsschritt** werden die Informationen über die Zusammenhänge der Vertices verwendet, um die innen liegenden Bereiche zu lokalisieren. Diese werden nach dem Zielraster, bspw. dem Bildschirm oder einfach einem definierten Speicherabschnitt (Framebuffer), diskretisiert. Hier wird die Baryzentrische Interpolation angewendet.

Dabei werden für alle innen liegenden Fragmente die bestehenden Vertexinformationen linear interpoliert.



Vereinfachte Renderpipeline – Phase 4

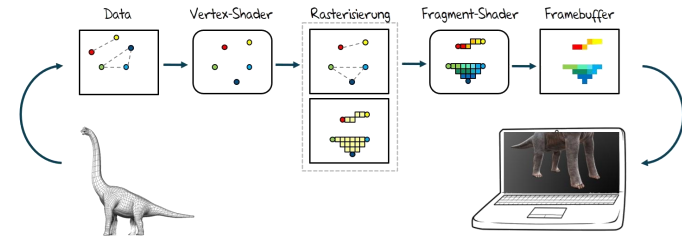
Wir haben in diesem Beispiel für die Vertexinformationen konkrete Farben verwendet. Nach dem Rasterisierungsprozess liegen nun für alle Fragmente interpolierte Werte vor. Der **Fragment-Shader** bestimmt parallel für jedes Fragment aus den interpolierten Werten eine Farbe. Wenn wir also einfach die interpolierten Farbwerte verwenden, sehen wir sehr schnell den auftretenden Effekt.



Vereinfachte Renderpipeline – Phase 5

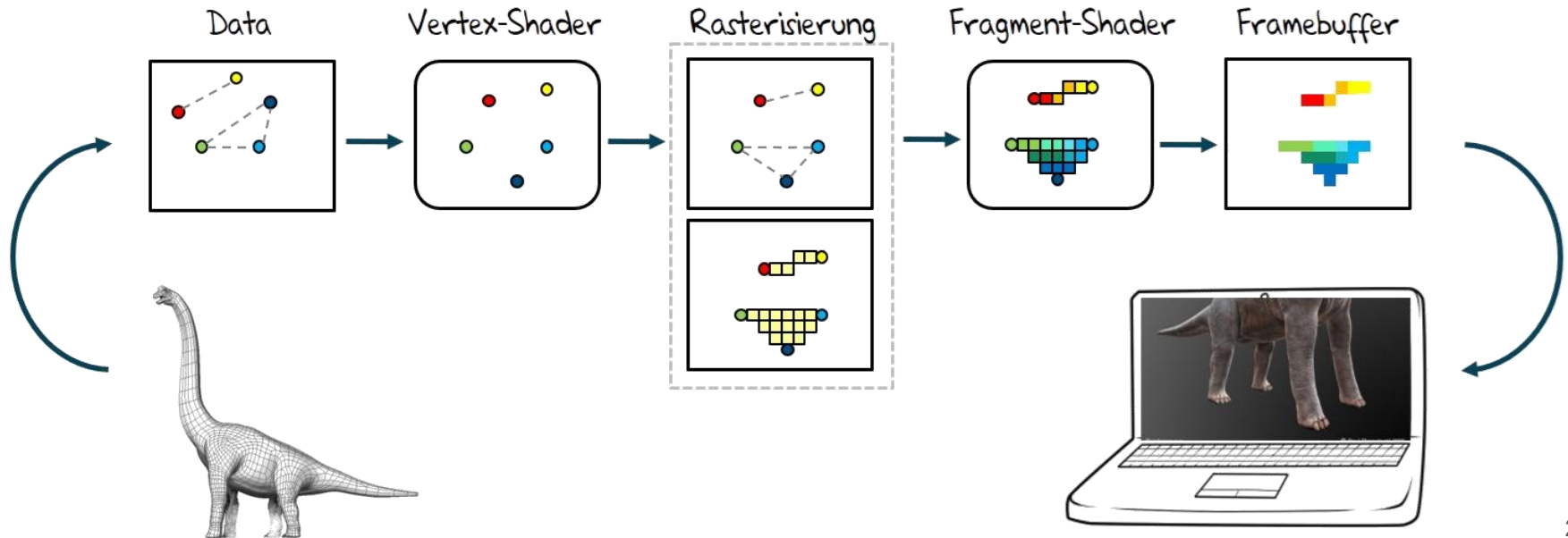
Beispielsweise könnten wir das erhaltene Raster, das wir in einem **Framebuffer** gesammelt haben, auf dem Bildschirm ausgeben und erhalten fließende Übergänge der Farben in den Primitiven.

Bei den abstrakten fünf Schritten der Rendering-Pipeline in OpenGL sehen wir die besondere Bereiche Vertex- und Fragment-Shader. Das sind genau die beiden Phasen, die wir später direkt manipulieren und selbst programmieren werden.



Vereinfachte Renderpipeline - Zusammenfassung

Typische Renderpipeline von OpenGL: Gegeben sind geometrische Primitive mit unterschiedlichen Informationen (links als Farben dargestellt). Anschließend werden diese durch den Vertex-Shader an die gewünschten, neuen Positionen transformiert. Nach dem Zielraster werden die Inhalte der Primitiven diskretisiert und die Inhalte linear interpoliert (mittig). Der Fragment-Shader übernimmt die Färbung der interpolierten Werte. Schließlich wählen wir den Ausgabebereich, z. B. den Bildschirm (rechts).





Schauen wir uns das Zusammenspiel von
Java und OpenGL an.

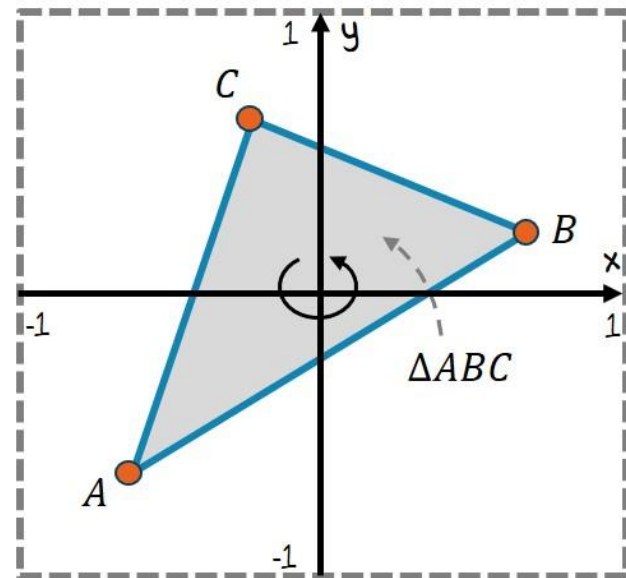
Ein Dreieck in OpenGL konstruieren

Für das folgende Beispiel werden wir Punkte im 3-dimensionalen Raum über die Funktion `glVertex3d(x,y,z)` beschreiben und drei dieser Punkte zu einem Dreieck-Objekt `GL_TRIANGLES` zusammenführen.

```
glBegin(GL_TRIANGLES);  
    glVertex3d(x1, y1, z1); // Punkt A  
    glVertex3d(x2, y2, z2); // Punkt B  
    glVertex3d(x3, y3, z3); // Punkt C  
glEnd();
```

Java

Die Punkte eines Dreiecks ΔABC werden **gegen den Uhrzeigersinn** angegeben. Wenn die Koordinaten x und y der angegebenen Punkte im Intervall von $[-1,1]$ liegen und die z -Komponente auf 0 gesetzt wird, beschreiben wir genau den sichtbaren Bereich im Ausgabebereich.



Dreieck ΔABC

Die Endung 'd' bei `glVertex3d` gibt an, dass die Parameter als Double-Werte eingegeben werden. Alternativ könnten wir auch ein 'f' für Float angeben, müssten dann allerdings darauf achten, die Parameter immer entsprechend korrekt zu casten.

Bewegtes Dreieck realisieren

Jetzt wollen wir dem Dreieck eine leicht **variierende Farbe** und etwas **Bewegung** geben. Dazu nutzen wir den Einfluss der harmonischen Sinus-Funktion in Abhängigkeit zur Zeit t , die kontinuierliche Werte zwischen -1 und 1 liefert.

```
// importieren ...
public class DreieckFarbeBewegung extends LWJGLBasisFenster {
    // Konstruktor ...

    @Override
    public void renderLoop() {
        while (!Display.isCloseRequested()) {
            double t = System.nanoTime() / 1e9;
            clearBackground(0.95f, 0.95f, 0.95f, 1.0f);
            glColor3d(0.35 + 0.30*Math.sin(t),
                    0.63 + 0.24*Math.sin(t),
                    0.73 + 0.22*Math.sin(t));
            glBegin(GL_TRIANGLES);
            glVertex3d(-0.5, -0.5 + 0.3*Math.sin(t), 0);
            glVertex3d( 0.5,  0.3*Math.sin(t*1.3), 0);
            glVertex3d( 0.3*Math.sin(t*2), 0.5, 0);
            glEnd();
            Display.update();
        }
    }
    // main ...
}
```

Java

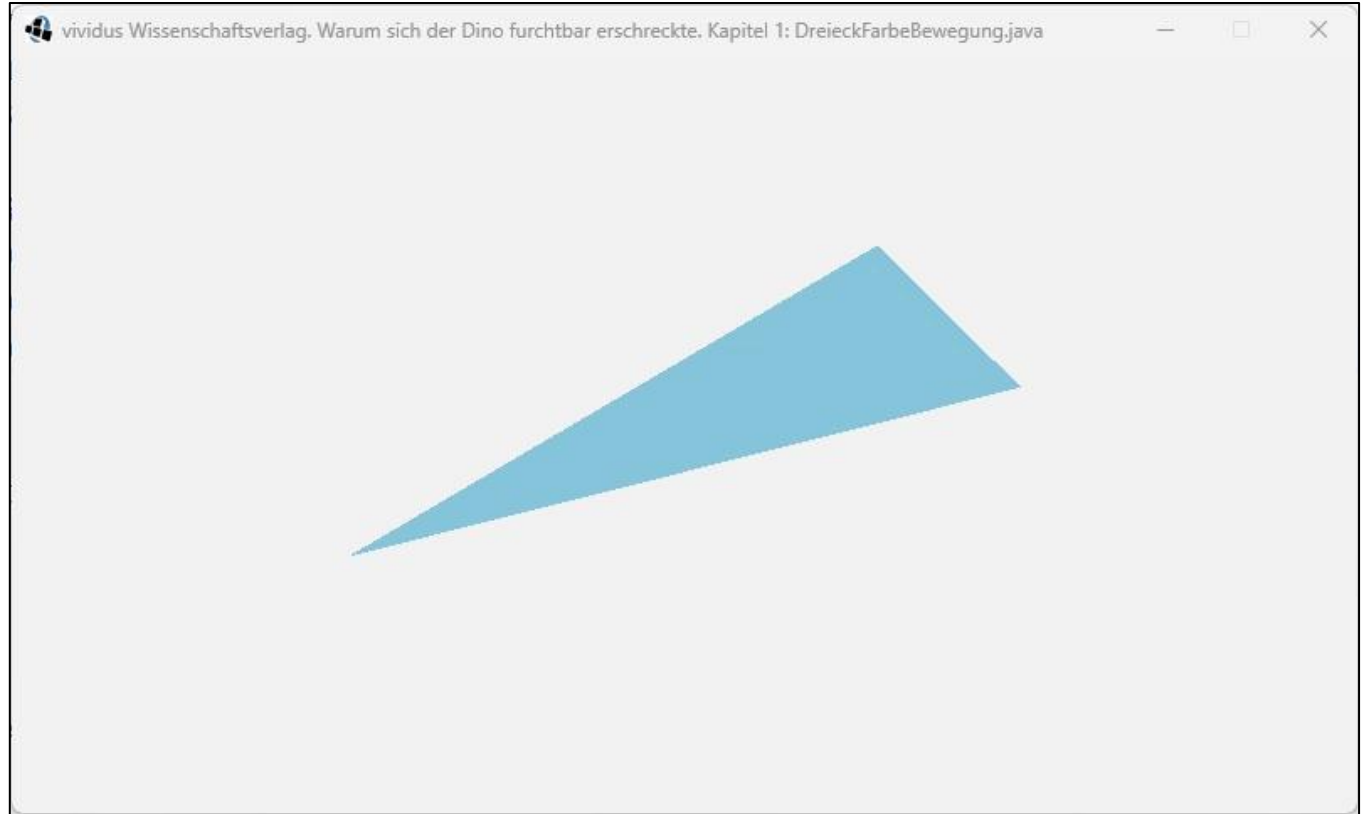
Die Systemzeit lassen wir uns durch die Methode **nanoTime** in Nanosekunden geben und rechnen sie in Sekunden um.

Durch den Befehl **glClearColor(r,g,b,a)** setzen wir die Farbe, mit der wir durch **glClear** den Hintergrund überschreiben wollen.

Wir erinnern uns daran, dass OpenGL eine Zustandsmaschine ist. Um für alle nachfolgenden Zeichenoperationen eine neue Farbe zu wählen, müssen wir mit **glColor3d(r,g,b,a)** eine neue Farbe setzen.

Bewegtes Dreieck ausgeführt

Die Punktkoordinaten und die Farbe des Dreiecks variieren.





Wie sah eigentlich nochmal die
Klasse **LWJGLBasisFenster** aus?

LWJGLBasisFenster und initDisplay

Die Methode `initDisplay()` wird bei jedem Programmstart ausgeführt und liefert das Display. Damit wir die Visualisierung umleiten können, z. B. in ein Canvas-Objekt, müssen wir die Methode von außen aufrufen können.

```
import org.lwjgl.LWJGLException;
import org.lwjgl.opengl.Display;
import org.lwjgl.opengl.DisplayMode;

public abstract class LWJGLBasisFenster {
    private int WIDTH, HEIGHT;
    private String TITLE;

    public LWJGLBasisFenster() {
        this("LWJGLBasisFenster", 640, 480);
    }

    public LWJGLBasisFenster(String title, int width, int height) {
        WIDTH = width;
        HEIGHT = height;
        TITLE = title;
    }
}
```

Java

```
public void initDisplay() {
    try {
        Display.setDisplayMode(new DisplayMode(WIDTH, HEIGHT));
        Display.setTitle(TITLE);
        Display.create();
    } catch (LWJGLException e) {
        e.printStackTrace();
    }
}

public abstract void renderLoop();

public void start() {
    initDisplay();
    renderLoop();
    Display.destroy();
    System.exit(0);
}
}
```

Java

LWJGLBasisFenster anpassen

Für ein kleines Beispiel wollen wir die Initialisierung des Displays von der Erzeugung eines Fensterexemplars trennen. Das erleichtert uns die Arbeit mit den nachfolgenden Beispielen, da nur noch die abstrakte Methode **renderLoop** implementiert werden muss.

```
public abstract class LWJGLBasisFenster {  
    private int WIDTH, HEIGHT;  
    private String TITLE;  
  
    public LWJGLBasisFenster() {  
        this("LWJGLBasisFenster", 640, 480);  
    }  
  
    public LWJGLBasisFenster(int width, int height) {  
        this("LWJGLBasisFenster", width, height);  
    }  
  
    public LWJGLBasisFenster(String title, int width, int height) {  
        WIDTH = width; HEIGHT = height; TITLE = title;  
    }  
  
    public void initDisplay() {  
        try {  
            Display.setDisplayMode(new DisplayMode(WIDTH, HEIGHT));  
            Display.setTitle(TITLE); Display.create();  
        } catch (LWJGLException e) {  
            e.printStackTrace();  
        }  
    }  
}
```

Java

```
public void initDisplay(Canvas c) {  
    try {  
        Display.setParent(c);  
    } catch (LWJGLException e) {  
        e.printStackTrace();  
    }  
    initDisplay();  
}  
  
public abstract void renderLoop();  
  
public void start() {  
    renderLoop();  
    Display.destroy();  
    System.exit(0);  
}  
}
```

Java

Jetzt können wir die LWJGL-Ausgabe umleiten!

AWT/Swing und LWJGL zusammenbringen

Jetzt können wir die Interaktion mit Swing-Elementen, wie z. B. mit einem Toggle-Button, und dem LWJGL-Display zeigen. Über einen Button können wir jetzt die Hintergrundfarbe zwischen Rot und Blau ändern.

```
// importieren ...
```

Java

```
public class HintergrundUmschalten extends LWJGLBasisFenster {
    private JToggleButton tb;

    public HintergrundUmschalten() {
        super(640, 480);
        JFrame f = new JFrame();
        tb = new JToggleButton("umschalten");
        f.add(tb, BorderLayout.SOUTH);
        Canvas c = new Canvas();
        f.add(c);
        f.setBounds(0, 0, 640, 480 + tb.getHeight());
        f.setTitle("Hintergrund umschalten");
        f.setLocationRelativeTo(null);
        f.setVisible(true);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        initDisplay(c);
    }
}
```

```
@Override
```

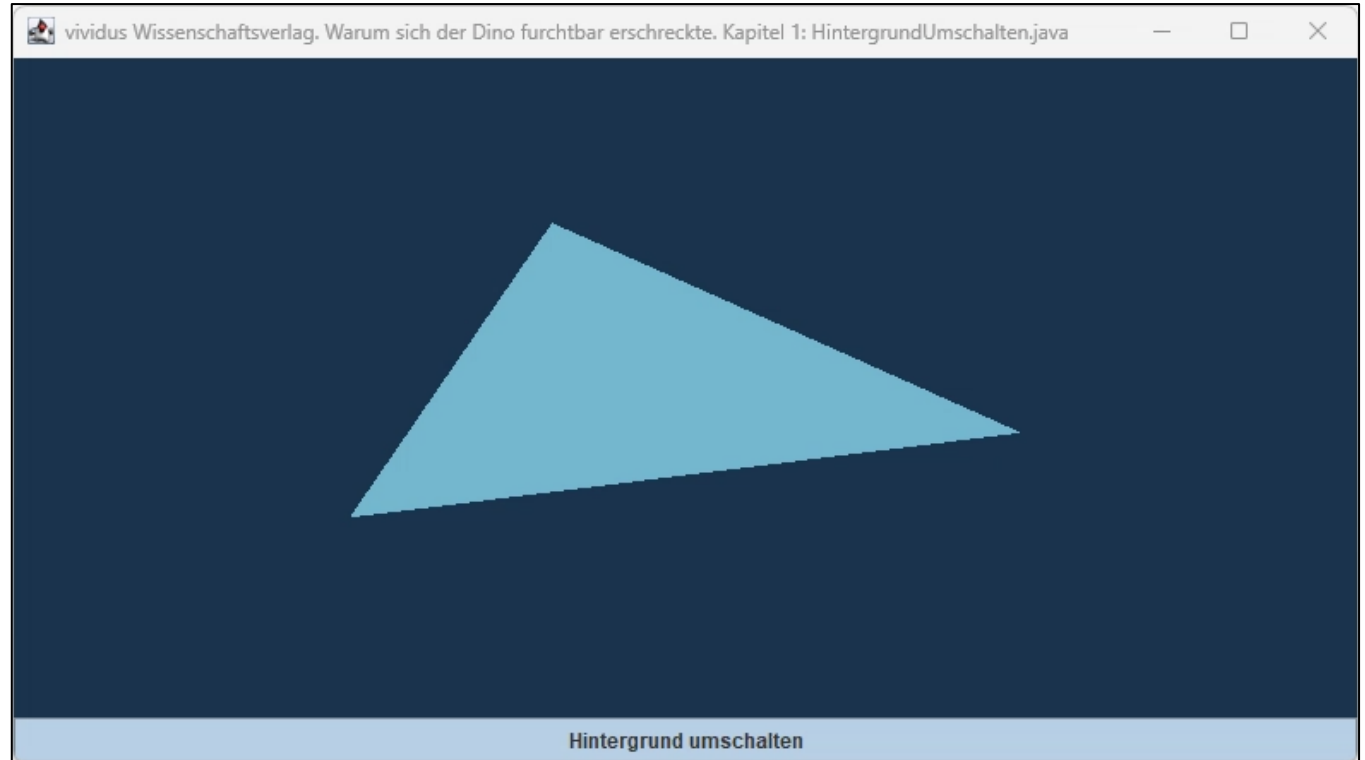
Java

```
public void renderLoop() {
    while (!Display.isCloseRequested()) {
        double t = System.nanoTime()/1e9;
        if (tb.isSelected())
            glClearColor(0.1f, 0.2f, 0.3f, 1);
        else
            glClearColor(1, 0, 0, 1);
        glClear(GL_COLOR_BUFFER_BIT);
        glColor3d(0, .7 + 0.3 * Math.sin(t), 0);
        glBegin(GL_TRIANGLES);
            glVertex3d(-.5f, -.5f + 0.3 * Math.sin(t), 0);
            glVertex3d(.5f, 0 + 0.3 * Math.sin(t*1.3), 0);
            glVertex3d(+0.3 * Math.sin(t*2), .5f, 0);
        glEnd();
        Display.update();
    }
}

// main ...
}
```

Hintergrund mit ToggleButton umschalten

Durch den Toggle-Button können wir die Hintergrundfarbe ändern.



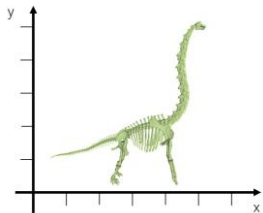


Kleine Einführung in Transformationen ...

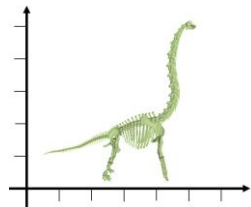
Geometrische Transformationen

Mit geometrischen Transformationen bezeichnen wir das Verschieben ([Translation](#)), das Drehen ([Rotation](#)), das Spiegeln an einem Punkt oder einer Geraden ([Reflexion](#)) und das Verkleinern bzw. Vergrößern ([Skalierung](#)) von Objekten innerhalb eines Koordinatensystems oder sogar zwischen verschiedenen Koordinatensystemen.

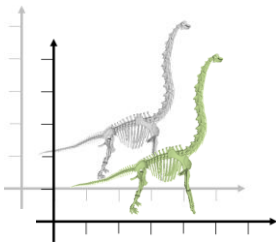
Objekt



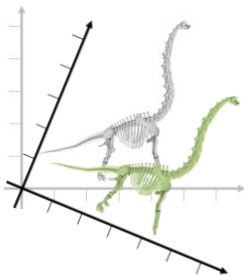
Identität



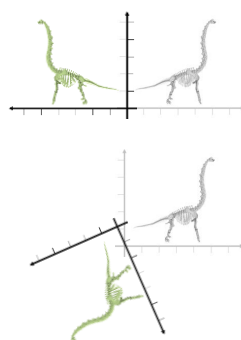
Translation



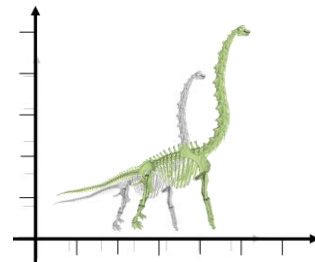
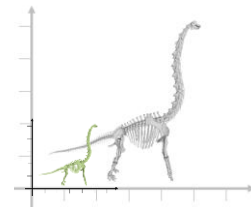
Rotation



Reflexion



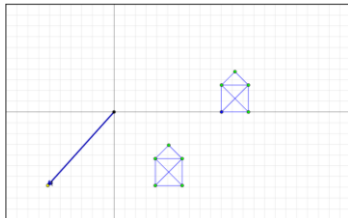
Skalierung



Geometrische Transformationen in 2D (starre)

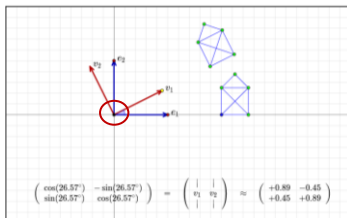
Schauen wir uns die **nicht-formverändernden (starken) Transformationen** an (interaktiv bei Mathe Vital [1]).

Translation (Verschiebung)



$$\begin{bmatrix} v_x' \\ v_y' \end{bmatrix} = \begin{bmatrix} v_x \\ v_y \end{bmatrix} + \begin{bmatrix} t_x \\ t_y \end{bmatrix} = \begin{bmatrix} v_x + t_x \\ v_y + t_y \end{bmatrix}$$

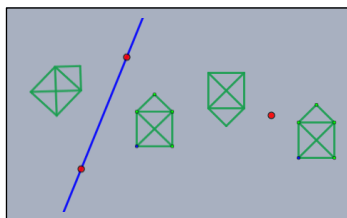
Rotation (Drehung)



$$\begin{bmatrix} v_x' \\ v_y' \end{bmatrix} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \cdot \begin{bmatrix} v_x \\ v_y \end{bmatrix} = \begin{bmatrix} av_x + bv_y \\ cv_x + dv_y \end{bmatrix}$$

$$R = \begin{bmatrix} a & b \\ c & d \end{bmatrix} = \begin{bmatrix} \cos(\alpha) & -\sin(\alpha) \\ \sin(\alpha) & \cos(\alpha) \end{bmatrix}$$

Reflexion (Spiegelung)



$$\begin{bmatrix} v_x' \\ v_y' \end{bmatrix} = \begin{bmatrix} v_x \\ -v_y \end{bmatrix}$$

Spiegelung an Geraden

$$\begin{bmatrix} v_x' \\ v_y' \end{bmatrix} = R \begin{bmatrix} v_x \\ v_y \end{bmatrix}$$

Spiegelung an Punkten

Starre (nicht formverändernde) Transformationen

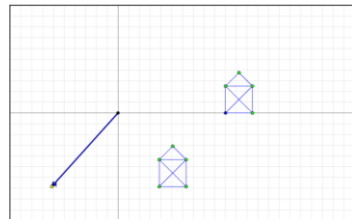
Jede starre Transformation (Rigid-Body-Transformation) lässt sich aus **Translationen**, **Rotationen** und **Reflexionen** zusammensetzen und verändert die Form des Objekts nicht. Die Winkel bleiben erhalten. Die Reihenfolge ist hier entscheidend, da assoziativ aber nicht kommutativ!

[1] <http://www.mathe-vital.de/Indras/1-1.html>

Geometrische Transformationen in 2D (affine)

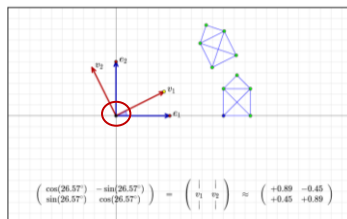
Jetzt erweitern wir die geometrische zu den **affinen Transformationen** (interaktiv bei Mathe Vital [1]).

Translation (Verschiebung)



$$\begin{bmatrix} v_x' \\ v_y' \end{bmatrix} = \begin{bmatrix} v_x \\ v_y \end{bmatrix} + \begin{bmatrix} t_x \\ t_y \end{bmatrix} = \begin{bmatrix} v_x + t_x \\ v_y + t_y \end{bmatrix}$$

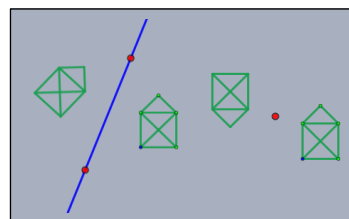
Rotation (Drehung)



$$\begin{bmatrix} v_x' \\ v_y' \end{bmatrix} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \cdot \begin{bmatrix} v_x \\ v_y \end{bmatrix} = \begin{bmatrix} av_x + bv_y \\ cv_x + dv_y \end{bmatrix}$$

$$R = \begin{bmatrix} a & b \\ c & d \end{bmatrix} = \begin{bmatrix} \cos(\alpha) & -\sin(\alpha) \\ \sin(\alpha) & \cos(\alpha) \end{bmatrix}$$

Reflexion (Spiegelung)



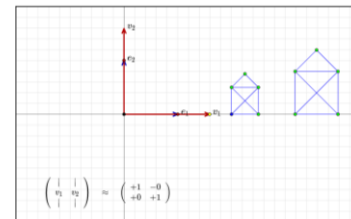
$$\begin{bmatrix} v_x' \\ v_y' \end{bmatrix} = \begin{bmatrix} v_x \\ -v_y \end{bmatrix}$$

Spiegelung an Geraden

$$\begin{bmatrix} v_x' \\ v_y' \end{bmatrix} = R \begin{bmatrix} v_x \\ v_y \end{bmatrix}$$

Spiegelung an Punkten

Skalierung (Streckung/Stauchung)



$$\begin{bmatrix} v_x' \\ v_y' \end{bmatrix} = s \cdot \begin{bmatrix} v_x \\ v_y \end{bmatrix} = \begin{bmatrix} sv_x \\ sv_y \end{bmatrix}$$

Gleichförmige Skalierung (uniform)

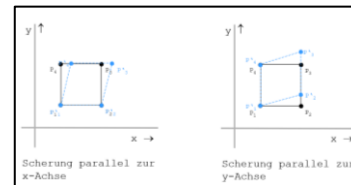
$$\begin{bmatrix} v_x' \\ v_y' \end{bmatrix} = \begin{bmatrix} s_x \\ s_y \end{bmatrix} \odot \begin{bmatrix} v_x \\ v_y \end{bmatrix} = \begin{bmatrix} s_x v_x \\ s_y v_y \end{bmatrix}$$

Ungleichförmige Skalierung (nicht-uniform)

Affine Transformationen

Jede affine Transformation (Ähnlichkeitstransformationen) lässt sich aus **Translationen**, **Rotationen**, **Skalierungen** und **Scherungen** zusammensetzen. Die Reihenfolge ist hier entscheidend, da assoziativ aber nicht kommutativ!

Scherung



$$\begin{bmatrix} v_x' \\ v_y' \end{bmatrix} = \begin{bmatrix} v_x + a_x y \\ a_y x + v_y \end{bmatrix}$$

[1] <http://www.mathe-vital.de/Indras/1-1.html>

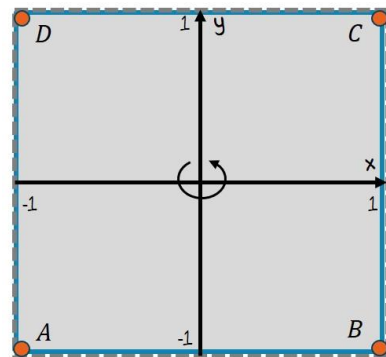
[2] <https://www.matheretter.de/wiki/transformationen-2d-matrizen>

Abfolge von Transformationsketten

Obwohl die notwendige Mathematik für eine [Verkettung von Transformationen](#) erst später vorgestellt wird, werden wir ins kalte Wasser springen und ein einfaches Viereck für die Demonstration verschiedener Transformationen verwenden:

```
glBegin(GL_QUADS);  
    glVertex3f(-1, -1, 0); // Punkt A  
    glVertex3f(1, -1, 0);  // Punkt B  
    glVertex3f(1, 1, 0);   // Punkt C  
    glVertex3f(-1, 1, 0);  // Punkt D  
glEnd();
```

Java



Viereck ABCD

Wir müssen an dieser Stelle verstehen, dass die Reihenfolge der angegebenen Transformationsanweisungen in OpenGL [rückwärts](#) ausgeführt wird. Wenn wir beispielsweise die Transformationskette $M = IPTRS$ erzeugen wollen, dann müssen wir darauf achten, dass wir die Reihenfolge der Befehle umdrehen:

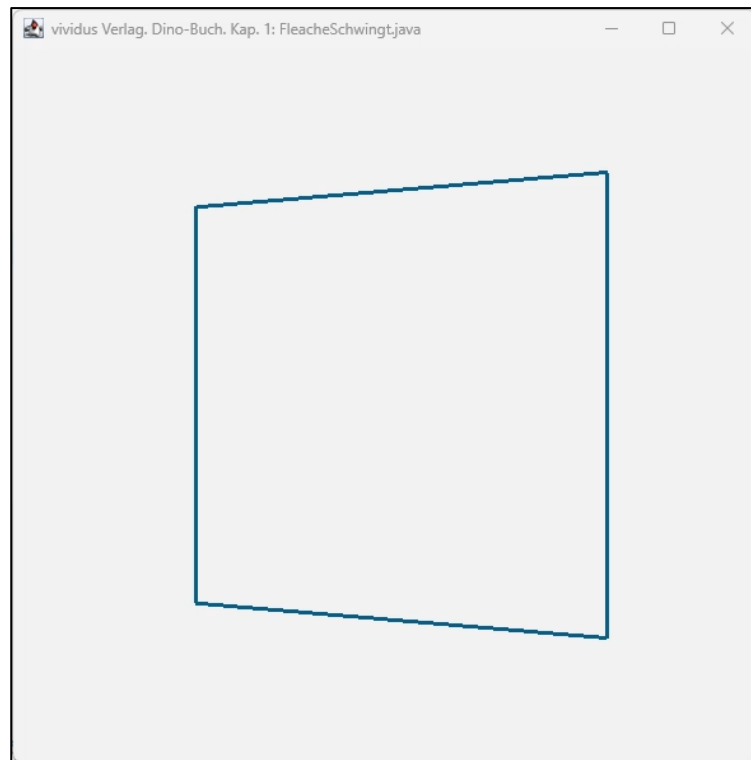
```
glLoadIdentity();           // M=I  
glFrustum(-1, 1, -1, 1, 3, 10); // M*P  
glTranslated(0, 0, -4);     // M*T  
glRotatef((float)Math.sin(t) * 120, 0, 1, 0); // M*R  
glScaled(.5, .5, .5);       // M*S  
renderFlaeche(); // Object-Space
```

Hier sind S eine Skalierungsmatrix, R eine Rotationsmatrix, T eine Translationsmatrix, P eine Projektionsmatrix und I die Identitätsmatrix. Die neue Matrix wird von rechts heran multipliziert, daher wird zuerst die Skalierung durchgeführt. Das ist wichtig, denn die [Matrixmultiplikation ist nicht kommutativ](#)!

Beispiel schwingende Fläche

Hier sehen wir das bewegende Quad, das in jedem Schritt folgende Transformationen durchläuft:

```
glLoadIdentity();           // M=I
glFrustum(-1, 1, -1, 1, 3, 10); // M*P
glTranslated(0, 0, -4);      // M*T
glRotatef((float)Math.sin(t) * 120, 0, 1, 0); // M*R
glScaled(.5, .5, .5);        // M*S
renderFlaeche(); // Object-Space
```





Wir entwickeln uns parallel zu den Beispielen eine nützliche Sammlung: Primitives for OpenGL ([POGL](#))

Primitives für OpenGL (POGL)

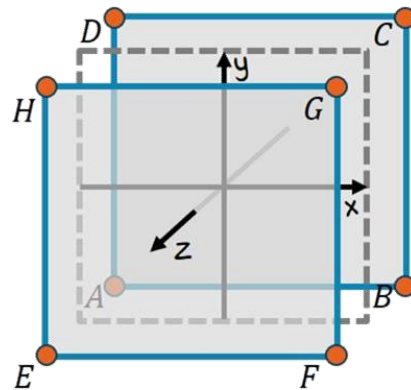
In **POGL** wollen wir OpenGL-Makro-Befehle konstruieren und sammeln. Diese sollen uns lediglich zur Verschlankung des Codes und Vermeidung von unnötigem, redundantem Code dienen:

```
public static void clearBackgroundWithColor(float r, float g, float b, float a) {  
    glClearColor(r, g, b, a);  
    glClear(GL_COLOR_BUFFER_BIT);  
}
```

Schauen wir uns kurz den Beispielcode für einen 3D-OpenGL-Würfel an, der aus sechs Vierecken besteht:

```
glBegin(GL_QUADS);  
    glVertex3f(-1, -1, -1); // Punkt A  
    glVertex3f(1, -1, -1); // Punkt B  
    glVertex3f(1, 1, -1); // Punkt C  
    glVertex3f(-1, 1, -1); // Punkt D  
  
    glVertex3f(-1, -1, 1); // Punkt E  
    glVertex3f(1, -1, 1); // Punkt F  
    glVertex3f(1, 1, 1); // Punkt G  
    glVertex3f(-1, 1, 1); // Punkt H  
  
    glVertex3f(-1, -1, -1);  
    glVertex3f(-1, 1, -1);  
    glVertex3f(-1, 1, 1);  
    glVertex3f(-1, -1, 1);
```

```
    glVertex3f(1, -1, -1);  
    glVertex3f(1, 1, -1);  
    glVertex3f(1, 1, 1);  
    glVertex3f(1, -1, 1);  
  
    glVertex3f(-1, -1, -1);  
    glVertex3f(1, -1, -1);  
    glVertex3f(1, 1, 1);  
    glVertex3f(-1, -1, 1);  
  
    glVertex3f(-1, 1, -1);  
    glVertex3f(1, 1, -1);  
    glVertex3f(1, 1, 1);  
    glVertex3f(-1, 1, 1);  
  
glEnd();
```



Konstruktion Würfel

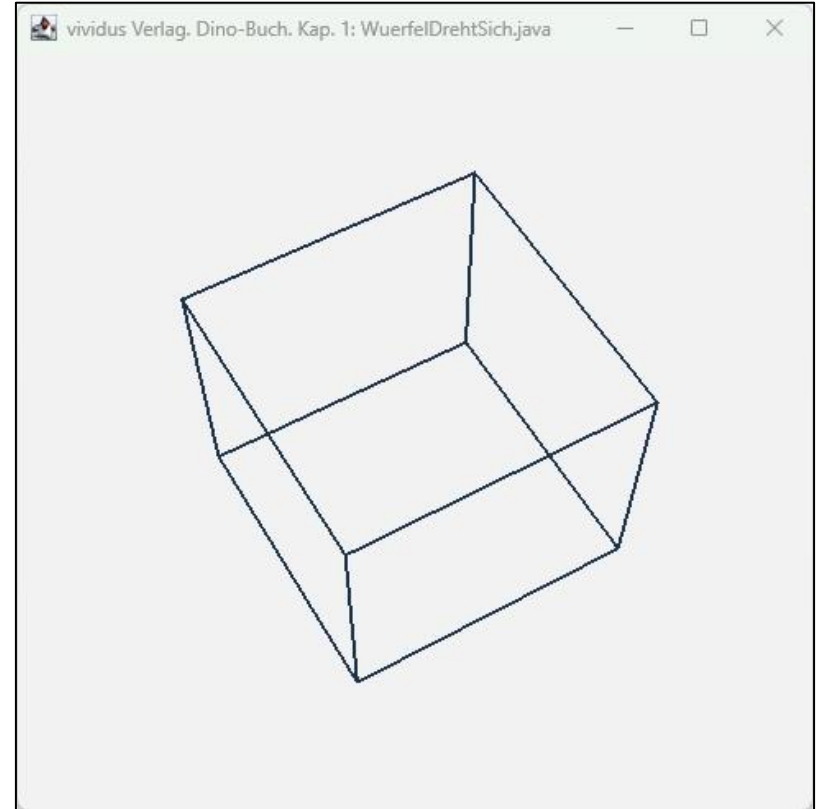
Nutzung von POGL

Da die Klasse POGL selbst nur **statische Methoden** anbietet, können diese direkt ohne Konstruktor aufgerufen werden. Im nachfolgenden Beispiel werden wir einen Würfel zeichnen und diesen rotieren lassen:

```
double t = (System.nanoTime()-start) / 1e9;
POGL.clearBackgroundWithColor(1, 1, 1, 1);

glLoadIdentity();
glFrustum(-1, 1, -1, 1, 4, 10);
glTranslated(0, 0, -5);
glRotatef((float) t * 100, 1, 0, 0);
glRotatef((float) t * 100, 0, 1, 0);
glScaled(.5, .5, .5);

glColor3d(0, 0, 0);
POGL.renderWuerfel();
```





Jetzt wollen wir ein paar anspruchsvollere
OpenGL-Beispiele realisieren

Würfelprojektion auf eine Fläche

Was sich alles mit den verschiedenen Transformationen anstellen lässt, soll folgendes Beispiel kurz zeigen. Wir wollen den rotierenden Würfel aus dem vorhergehenden Beispiel auf eine rotierende Fläche projizieren. Da alle Transformationen **rückwärts** laufen, müssen wir diesen also eigentlich **von unten nach oben lesen**:

```
glLoadIdentity();
glFrustum(-1, 1, -1, 1, 3, 10);
glTranslated(0, 0, -4);
glRotatef((float) Math.sin(t) * 120, 0, 1, 0);
glScaled(.5, .5, .5);

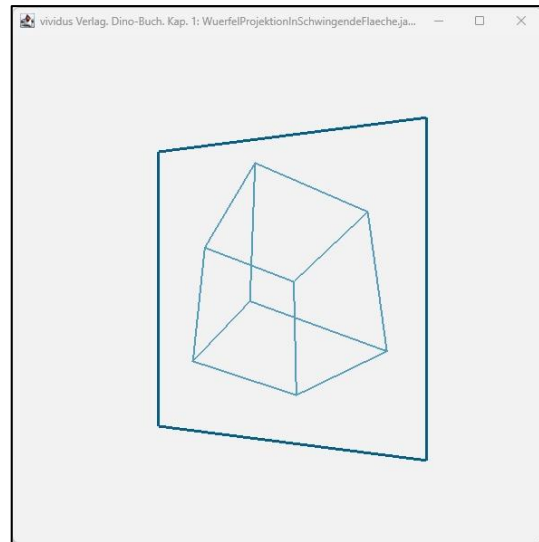
renderFlaeche(); // Object-Space

// Innerhalb dieser Projektionsfläche wollen wir eine
// neue Szene konstruieren, also eine Art Kamerabildfläche
// dazu müssen wir den Projektionsbereich zunächst
// abflachen, also die z-Koordinate auf 0 bringen
glScaled(1, 1, 0);

glFrustum(-1, 1, -1, 1, 3, 5);
glTranslated(0, 0, -5);
glRotatef((float) t * 100, 1, 0, 0);
glRotatef((float) t * 100, 0, 1, 0);
glScaled(.5, .5, .5);

renderWuerfel(); // Object-Space
```

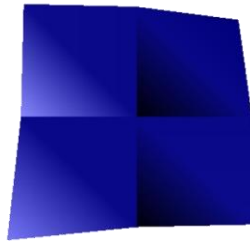
Die Projektionsfläche rotiert und zeigt innerhalb dieser Fläche einen rotierenden Würfel.



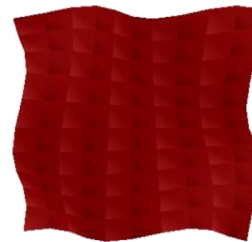
Wehende Fläche

Zu guter letzt werden wir noch mehr Dynamik in unsere Quads bringen und sogar die Seiten animieren (zumindest gaukeln wir das vor). Es soll eine Fahne realisiert werden, mit gleichmäßigen, wellenförmigen Bewegungen an allen Seiten. Um das zu bewerkstelligen, zeichnen einfach sehr viele Quads nebeneinander.

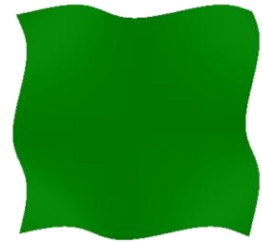
```
glBegin(GL_QUADS);  
double periode = 15, amplitude = 0.03, step = 0.01;  
float r = 0.0f, g = 0.5f, b = 0.0f;  
double bl, br, tl, tr;  
  
for (double y = -0.5 ; y < 0.5; y += step) {  
    for (double x = -0.5 ; x < 0.5; x += step) {  
        bl = Math.sin((t*10 + x*y*periode))*amplitude;  
        br = Math.sin((t*10 + (x+step)*y*periode))*amplitude;  
        tr = Math.sin((t*10 + (x+step)*(y+step)*periode))*amplitude;  
        tl = Math.sin((t*10 + x*(y+step)*periode))*amplitude;  
  
        glColor3d(r+bl, g+bl, b+bl);  
        glVertex2d(x+bl, y+bl);  
        glColor3d(r+br, g+br, b+br);  
        glVertex2d(x+step+br, y+br);  
        glColor3d(r+tr, g+tr, b+tr);  
        glVertex2d(x+step+tr, y+step+tr);  
        glColor3d(r+tl, g+tl, b+tl);  
        glVertex2d(x+tl, y+step+tl);  
    }  
}  
glEnd();
```



4 Vierecke



100 Vierecke



10.000 Vierecke

Links: Hier sehen wir das Ergebnis mit den Parametern **amplitude=0.05**, **step=0.5** und einem kleinen Flag, das die untere linke Ecke alternierend verändert. Dadurch wird die Konstruktion der Fläche klar. Mitte: **amplitude=0.03**, **step=0.1** Rechts: Die finale Visualisierung basiert einfach auf vielen Dreiecken, deren Farbwerte an den Rändern gleich sind. Wir verwenden hier die Parameter **amplitude=0.03** und **step=0.01**.

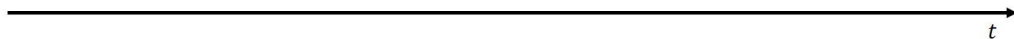
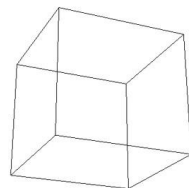
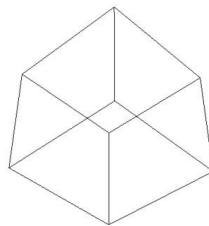
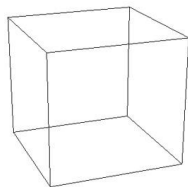
Nutzung von POGL

Da die Klasse POGL selbst nur **statische Methoden** anbietet, können diese direkt ohne Konstruktor aufgerufen werden. Im nachfolgenden Beispiel werden wir einen Würfel zeichnen und diesen rotieren lassen:

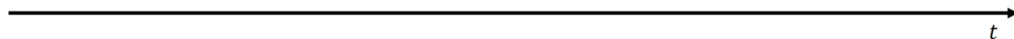
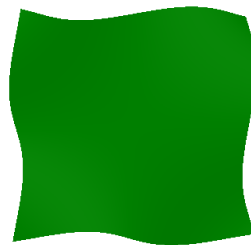
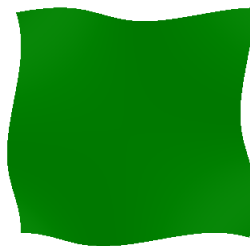
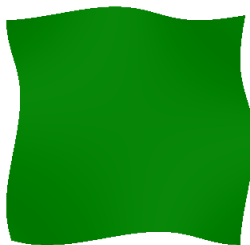
```
double t = (System.nanoTime()-start) / 1e9;
POGL.clearBackgroundWithColor(1, 1, 1, 1);

glLoadIdentity();
glFrustum(-1, 1, -1, 1, 4, 10);
glTranslated(0, 0, -5);
glRotatef((float) t * 100, 1, 0, 0);
glRotatef((float) t * 100, 0, 1, 0);
glScaled(.5, .5, .5);

glColor3d(0, 0, 0);
POGL.renderWuerfel();
```

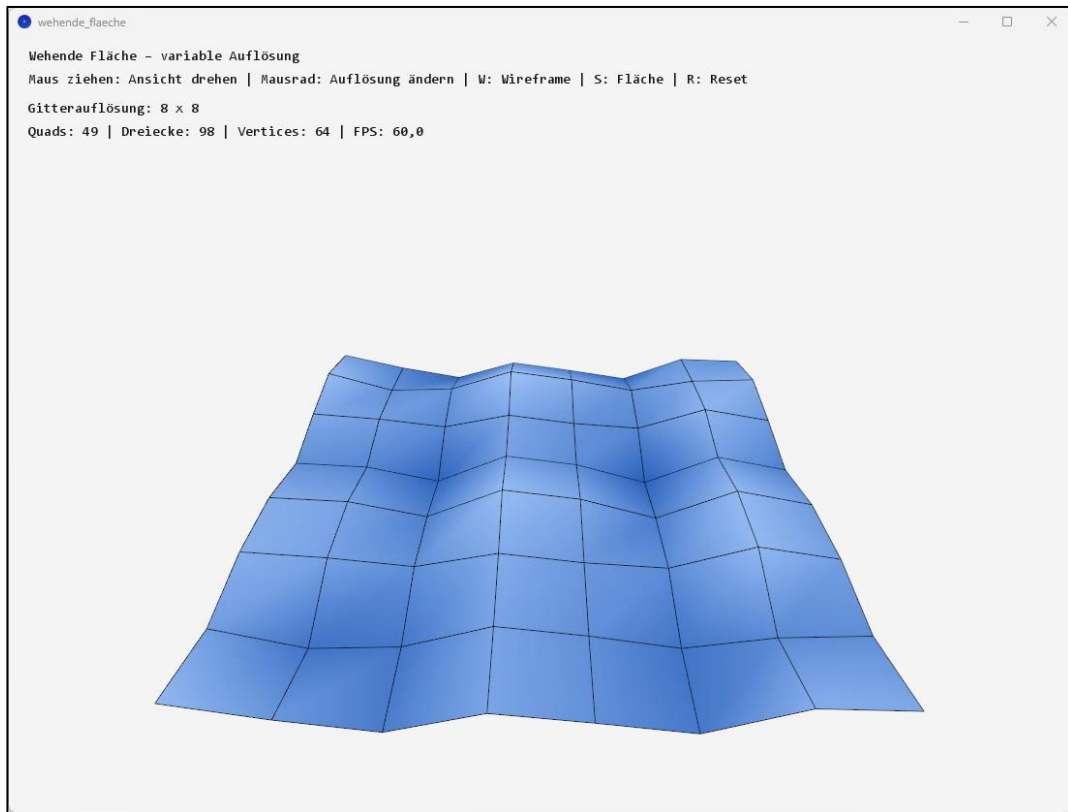


```
long startTime = System.nanoTime();
while (!Display.isCloseRequested()) {
    POGL.clearBackgroundWithColor(0.1f, 0.2f, 0.3f, 1);
    double t = (System.nanoTime()-startTime)/1e9;
    POGL.renderWehendeFlaeche(t);
    Display.update();
}
```



Interaktives Beispiel zur wehenden Fläche

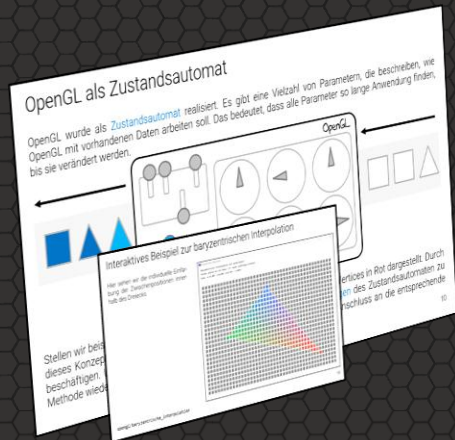
Hier sehen wir die wehende Fläche in Processing als interaktives Beispiel. Die Gitterauflösung ist variabel.



OpenGL-Grundlagen: Dreiecke, Interpolation und Rendering-Pipeline

Key Takeaways

- Grafik-APIs: OpenGL, DirectX und Vulkan steuern die GPU
- OpenGL eignet sich besonders gut für Lehre und plattformübergreifende Anwendungen
- Dreiecke sind das zentrale Grundprimitive
- Baryzentrische Koordinaten ermöglichen Interpolation innerhalb eines Dreiecks
- Rendering basiert auf einer Transformationspipeline
- Objekte werden im Object Space definiert und durch Transformationen positioniert
- Transformationen werden kumulativ angewendet
- Flächen können als diskrete Gitter modelliert werden



Fragestellungen zum Vorlesungsteil

Im Anschluss an diese Veranstaltung sollten Sie die folgenden Fragen sicher beantworten können:

- (1) Erläutern Sie die vereinfachte Renderpipeline von OpenGL.
- (2) Besuchen Sie die Webseite von Mathe Vital [1] und experimentieren Sie mit den dort vorgestellten Transformationen herum.
- (3) Konstruieren Sie drei konkrete Zahlenbeispiele auf einem karierten Blatt für die baryzentrische Interpolation und berechnen Sie jeweils einen innenliegenden Punkt (Fall Dreieck).



[1] <http://www.mathe-vital.de/Indras/>

Praktische Aufgaben

Folgende praktische Aufgaben sollten Sie jetzt selbstständig durchführen:

Aufgabe 1) [Java] In der Klasse **TriangleInterpolation** finden Sie vier unterschiedliche Lösungen für die baryzentrische Interpolation. Finden Sie durch geeignete JUnit-Tests heraus, ob alle vier Lösungen korrekt sind. Überlegen Sie sich ein geeignetes Verfahren, um die Performance der vier Lösungen zu vergleichen.

Aufgabe 2) [Java] Implementieren Sie die baryzentrische Interpolation in einer interaktiven GUI in Java. Setzen Sie drei Punkte (größere Kreise) zufällig auf den Bildschirm. Achten Sie darauf, dass die drei Punkte ein Dreieck aufspannen. Jeder Punkt repräsentiert einen unterschiedlichen Farbwert. Jetzt kann die Maus einen Punkt innerhalb des Dreiecks anklicken und es erscheint ein neuer Punkt (größerer Kreis), der mit dem durch die baryzentrische Interpolation ermittelten Farbwert gefüllt ist.



Vielen Dank für die Aufmerksamkeit